

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE CODE: CSA4301

COURSE NAME: Internet Programming

Name: Disha P

Reg no: 192411086

CO-1: Develop well-structured, standards-compliant web pages using HTML/XHTML and XML, and apply Internet protocols and HTTP communication mechanisms for web content delivery.

1.HTTP Response Analysis

Question:

A web server returns the following HTTP response header:

HTTP/1.1 200 OK

Date: Mon, 01 Apr 2026 10:00:00 GMT

Content-Type: text/html

Content-Length: -150

Connection: keep-alive

Answers:

1. Identify any incorrect or invalid fields in the header.

- The **Content-Length: -150** field is invalid because Content-Length cannot be negative.

2. Analyze the issue with Content-Length and explain its impact.

- Content-Length specifies the size of the response body in bytes.
- A negative value is invalid and may cause browsers to fail to load the page correctly or terminate the connection.

3. Determine whether the status code matches the response correctness.

- **No.** The status code **200 OK** indicates success, but the invalid Content-Length makes the response malformed.

4. Suggest corrections for the identified errors.

HTTP/1.1 200 OK

Date: Wed, 01 Apr 2026 10:00:00 GMT

Content-Type: text/html

Content-Length: 150

Connection: keep-alive

2. DOM Structure Analysis

Question:

Consider the following DOM tree representation:

```
<html>
<head>
<title>Test Page</title>
</head>
<body>
<div>
<p>Hello World
</div>
</body>
</html>
```

Answers:

1. Identify missing or improperly closed elements.

- The closing tag is missing.

2. Analyze how the browser will interpret this structure.

- The browser automatically closes the <p> tag before the </div> tag and renders the page.

3. Determine the impact on rendering and layout.

- The page displays correctly in most browsers but fails HTML validation and may affect CSS or JavaScript behavior.

4. Suggest corrections to make the DOM valid.

```
<html>
<head>
<title>Test Page</title>
</head>
<body>
<div>
<p>Hello World</p>
</div>
</body>
</html>
```

3. GET vs POST in Login System

Question:

A login system uses two different methods:

Method	URL Example	Data Visibility	Security Level
GET	/login?user=admin&pass=1234	Visible in URL	Low
POST	/login	Hidden in body	Moderate

Answers:

1. Interpret the differences in data transmission between GET and POST.

- **GET:** Sends data through the URL, making it visible.
- **POST:** Sends data in the request body, making it hidden from the URL.

2. Analyze which method is more suitable for login and why.

- **POST** is more suitable because it hides user credentials from the URL and offers better security.

3. Identify security risks in using GET for authentication.

- Password is visible in the URL.
- Stored in browser history.
- Logged by servers.
- Can be bookmarked or shared accidentally.

4. Suggest best practices for secure login implementation.

- Use the **POST** method.
- Enable **HTTPS**.
- Hash passwords (e.g., bcrypt).
- Validate user input.
- Use secure sessions and CSRF protection.

4. HTML Code Inspection

Question:

Analyze the following HTML snippet:

```
<!DOCTYPE html>
<html>
<head>
<title>Sample</title>
</head>
<body>
<h1>Welcome
<p>This is a paragraph

</body>
</html>
```

Answers:

1. Identify missing closing tags.

- Missing and tags.

2. Analyze issues with the tag.

- The tag is missing the **alt** attribute.

3. Determine how browsers will handle these errors.

- Browsers automatically close unclosed tags and render the page, but the HTML is invalid.

4. Suggest a corrected version of the code.

```
<!DOCTYPE html>
<html>
<head>
<title>Sample</title>
</head>
```

```
<body>
<h1>Welcome</h1>
<p>This is a paragraph</p>

</body>
</html>
```

5. Browser–Server Communication Flow

Question:

The following sequence describes a web request:

1. User enters a URL
2. DNS lookup occurs
3. HTTP request is sent
4. Server processes the request
5. Response is returned
6. Browser renders the page

Answers:

1. Interpret each step in the communication flow.

1. User enters the website URL.
2. DNS converts the domain name into an IP address.
3. Browser sends an HTTP request to the server.
4. Server processes the request.
5. Server returns an HTTP response.
6. Browser renders the webpage.

2. Identify where delays can occur.

- DNS lookup
- Network transmission
- Server processing
- Downloading resources
- Browser rendering

3. Analyze the role of DNS in the process.

- DNS translates the domain name into an IP address so the browser can locate the web server.

4. Suggest optimizations to improve performance.

- Use DNS caching.
- Enable browser caching.
- Compress files using Gzip/Brotli.
- Minify CSS and JavaScript.
- Optimize images.
- Use a Content Delivery Network (CDN).
- Reduce server response time.